

A Comparative Study of NLP Methods

Feature-Based · Transformer-Based · Prompt-Based

20 Newsgroups Dataset

Abdul Jamil Azizi

Master in IT Digitalization & Sustainability

Lucerne University of Applied Sciences and Arts

Module: Natural Language Processing

What Is This Project About?

01

TF-IDF & Word2Vec

Simple neural network (MLP)
trained with classical text
representations

02

BERT Fine-Tuning

Directly fine-tuned bert-base-
uncased for 20-class
classification

03

MLM Adaptation

Domain-adapted BERT via
masked language modeling
before classification

04

Llama-3 Prompting

Zero-shot & few-shot
evaluation using Meta-Llama-
3-8B-Instruct

+ Bonus: QLoRA Parameter-Efficient Fine-Tuning

The Dataset: 20 Newsgroups

9,051

Training Samples

2,263

Validation Samples

7,532

Test Samples

20 Topic Categories

alt.atheism

comp.graphics

comp.os.ms-windows

comp.sys.ibm.pc

comp.sys.mac

comp.windows.x

misc.forsale

rec.autos

rec.motorcycles

rec.sport.baseball

rec.sport.hockey

sci.crypt

sci.electronics

sci.med

sci.space

soc.religion.christian

talk.politics.guns

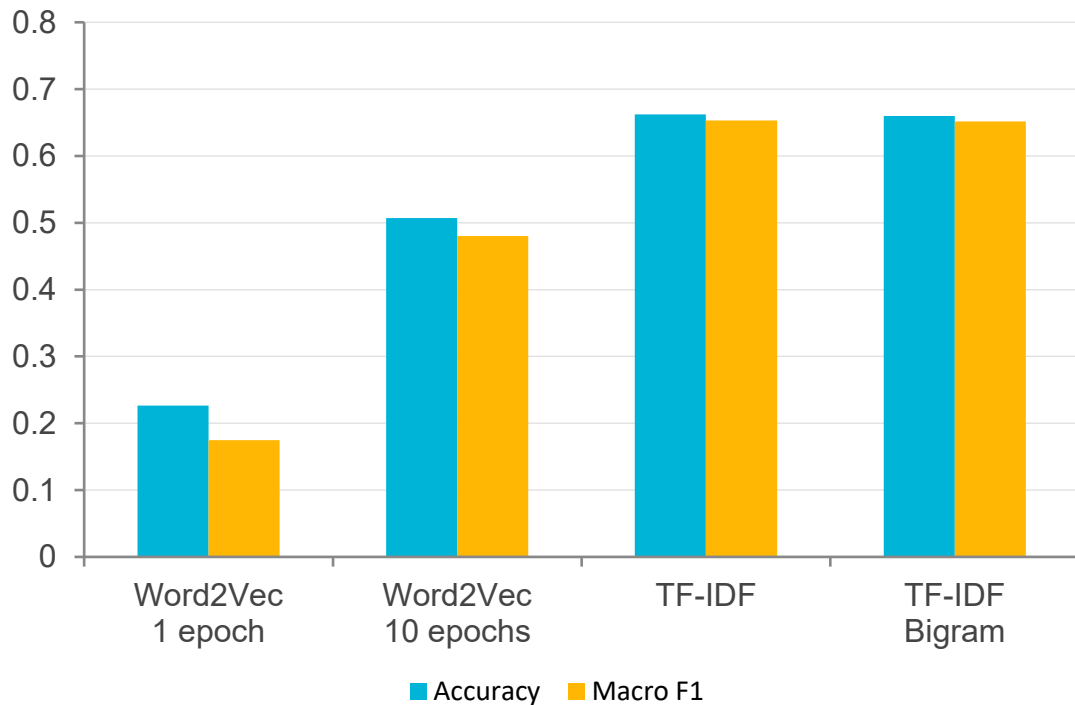
talk.politics.mideast

talk.politics.misc

talk.religion.misc

Part 1 — Classical Representations + MLP

Accuracy & Macro F1 by Method



Key Insights



TF-IDF is the clear winner — topic keywords carry strong class signal



Word2Vec jumped from 22% to 51% accuracy with 10× more training



Bigrams added negligible improvement over unigrams



MLP architecture stayed fixed — only the representation changed

Parts 2 & 3 — BERT: Direct vs. MLM-Adapted

Part 2 — Direct Fine-Tuning

- Model: bert-base-uncased
- Max seq length: 64 tokens
- Batch size: 8
- Learning rate: 2×10^{-5}
- Epochs: 2

Accuracy

0.6500

Macro F1

0.6300

Part 3 — MLM then Fine-Tuning

- Base model: bert-base-uncased
- MLM adaptation: 2,000 samples
- MLM epochs: 1
- Then same fine-tuning as Part 2
- Domain-aware BERT

MLM
boost

Accuracy

0.6600

Macro F1

0.6390

Part 4 — Llama-3: Zero-Shot vs Few-Shot

Zero-Shot

Model receives only:

- The list of 20 valid labels
- The input text
- Instruction to return one label

No examples provided.

12% **0.1040**

Accuracy

Macro F1

Few-Shot

Model receives:

- The list of 20 valid labels
- Labeled example pairs
- The input text

In-context learning — no weight updates.

50% **0.353**

Accuracy

Macro F1



Bonus — QLoRA: Parameter-Efficient Fine-Tuning

What is QLoRA?

LoRA injects trainable low-rank matrices into frozen transformer layers

QLoRA adds 4-bit quantization so large models fit on consumer GPUs

Only a fraction of parameters are trained — far cheaper than full fine-tuning

Ideal for running Llama-3 scale models on limited hardware

Experiment Setup

Model: Smaller instruct model

Epochs: 1

Environment: Kaggle GPU

Constraint: Limited memory

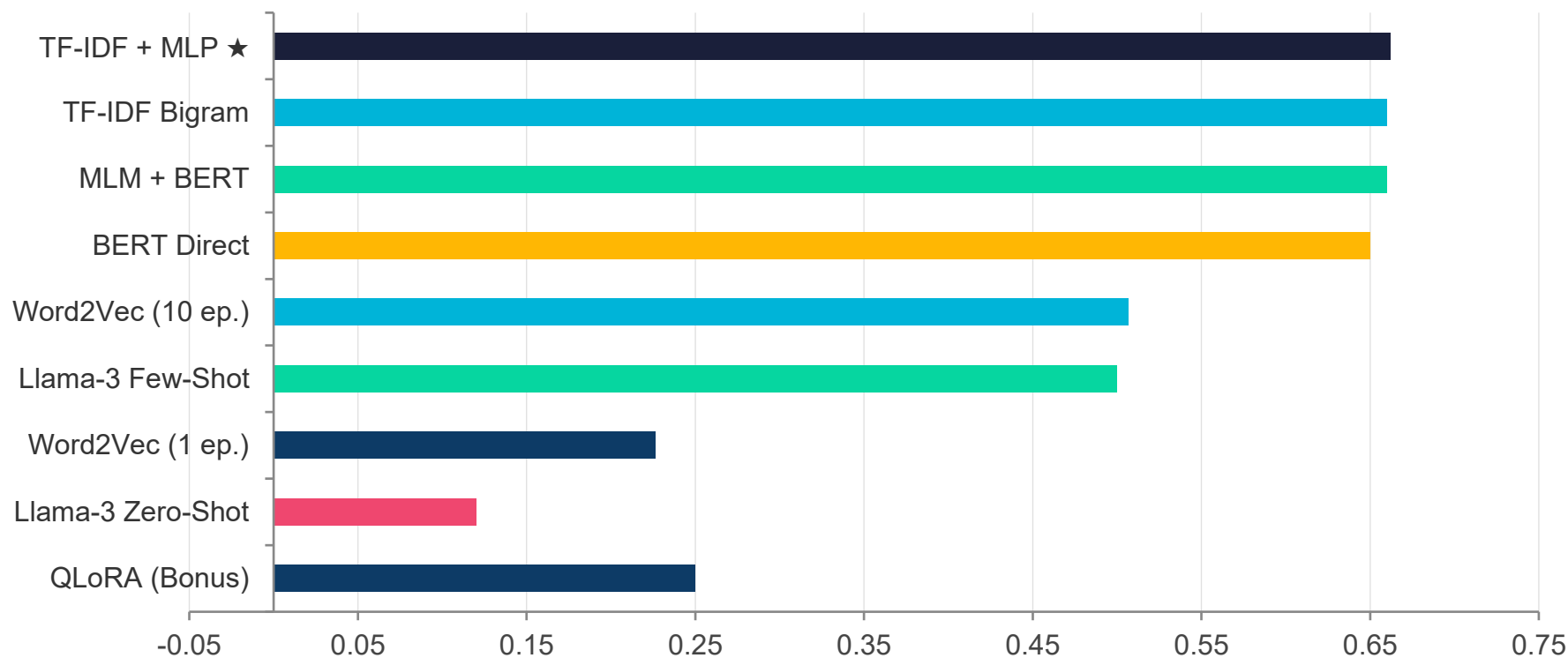
Result

25%
Accuracy

0.175
Macro F1

All Methods — Final Comparison

Accuracy by Method (higher = better)



Key Lessons from the Experiments

01

Right tool for the task

TF-IDF won not because it's the best model in general, but because it perfectly matches a topic-keyword-heavy dataset. Model complexity $\neq$ guaranteed accuracy.

02

Quality of representation matters

Word2Vec improved from 22% $\rightarrow$ 51% just by training longer. Embedding quality is as important as the classifier architecture.

03

Domain adaptation helps BERT

MLM pretraining on in-domain text gave BERT a small but consistent boost — even one epoch on 2,000 samples made a difference.

04

Examples unlock LLMs

Llama-3's accuracy jumped from 12% to 50% just by adding in-context examples. Prompt design is as powerful as model selection for inference-only use.

Conclusion

- TF-IDF + MLP achieved the best overall accuracy (66.2%) — simple but powerful for topic tasks
- BERT and MLM-BERT performed competitively, confirming transformer value in NLP
- Few-shot prompting showed the potential of LLMs without any training
- QLoRA demonstrated parameter-efficient fine-tuning is feasible under constraints
- Best method depends on task type, data, and available compute — not just model size